

Week 3: Seller and Inventory Management System

Scenario

The Organic Grocery Store gets its products from different suppliers. The system needs to store supplier details and keep track of the stock available for each product. This helps the store know which products are available and which products need to be restocked.

1. Create Seller and Inventory Tables

Seller Table

The **Seller table** stores information about the suppliers who provide products to the store.

```
SQL> CREATE TABLE Seller ( seller_id INT PRIMARY KEY, seller_name VARCHAR2(50), email VARCHAR2(100), phone VARCHAR2(15));  
Table created.
```

Inventory Table

The **Inventory table** stores the current stock quantity of each product.

```
SQL> CREATE TABLE Inventory (inventory_id INT PRIMARY KEY, product_id INT, stock_quantity INT, FOREIGN KEY (product_id) REFERENCES Product(product_id));  
Table created.
```

2. Establish Relationships

Seller and Product

A seller can supply **many products**.

For example, one seller may supply apples, bananas, and rice.

Relationship:

Seller → Product = One-to-Many (1 : N)

The seller_id connects the seller with the products they supply.

Product and Inventory

Each product has information about its available stock.

Relationship:

Product → Inventory = One-to-One (1 : 1)

The product_id is used to connect the Product table with the Inventory table.

3. Applying Constraints

Constraints – One Line Each

- **PRIMARY KEY:** Uniquely identifies each record in a table.
- **FOREIGN KEY:** Connects two tables.
- **UNIQUE:** Prevents duplicate values.
- **NOT NULL:** Does not allow an empty value.
- **CHECK:** Ensures the value follows a given condition.

Insert Values:

Insert values into Seller Table

```
SQL> INSERT ALL INTO Seller VALUES (1, 'Green Organic Store', 'green@gmail.com', '9876543210') INTO Seller VALUES (2, 'NatureFoods', 'nature@gmail.com', '9876543211') INTO Seller VALUES (3, 'Healthy Harvest', 'harvest@gmail.com', '9876543212') SELECT * FROM dual;
```

3 rows created.

```
SQL> SELECT * FROM Seller;
```

SELLER_ID	SELLER_NAME	EMAIL	P
1	Green Organic Store	green@gmail.com	
2	NatureFoods	nature@gmail.com	
3	Healthy Harvest	harvest@gmail.com	

Insert values into Inventory Table

```
SQL> INSERT ALL INTO Inventory VALUES (1, 101, 30) INTO Inventory VALUES (2, 102, 50) INTO Inventory VALUES (3, 103, 40) INTO Inventory VALUES (4, 104, 25) INTO Inventory VALUES (5, 105, 20) SELECT * FROM dual;
```

5 rows created.

```
SQL> SELECT * FROM Inventory;
```

INVENTORY_ID	PRODUCT_ID	STOCK_QUANTITY
1	101	30
2	102	50
3	103	40
4	104	25
5	105	20

4. Report 1: Seller-wise Product Report

This report shows the **seller name** and the **number of products supplied by each seller**.

```
SQL> SELECT s.seller_name, COUNT(p.product_id) AS total_products FROM Seller
s JOIN Product p ON s.seller_id = p.seller_id GROUP BY s.seller_name;
```

SELLER_NAME	TOTAL_PRODUCTS
Green Organic Store	2
NatureFoods	2
Healthy Harvest	1

This query connects the Seller and Product tables and counts how many products are supplied by each seller.

Report 2: Stock Availability Report

This report displays the **product name, available stock, and stock status**.

```
SQL> SELECT p.product_name, i.stock_quantity, CASE WHEN i.stock_quantity = 0
THEN 'Out of Stock' WHEN i.stock_quantity <= 10 THEN 'Low Stock' ELSE 'Avail
able' END AS stock_status FROM Product p JOIN Inventory i ON p.product_id =
i.product_id;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
--------------	----------------	--------------

Organic Apple	30	Available
Organic Banana	50	Available
Organic Carrot	40	Available
Brown Rice	25	Available
Green Tea	20	Available

The query checks the stock quantity and displays whether the product is **Available, Low Stock, or Out of Stock**.

Report 3: Inventory Analysis

Total Number of Products

```
SQL> SELECT COUNT(*) AS total_products FROM Inventory;
```

TOTAL_PRODUCTS
5

This counts the total number of products recorded in the inventory.

Products Out of Stock

```
SQL> SELECT COUNT(*) AS out_of_stock FROM Inventory WHERE stock_quantity = 0
;

OUT_OF_STOCK
-----
0
```

This finds the number of products that have **zero stock**.

Highest Stocked Product

```
SQL> SELECT p.product_name, i.stock_quantity FROM Product p JOIN Inventory i
ON p.product_id = i.product_id WHERE i.stock_quantity = (SELECT MAX(stock_q
uantity) FROM Inventory);

PRODUCT_NAME
-----
----
STOCK_QUANTITY
-----
Organic Banana
50
```

This identifies the product with the **highest available stock**.

Average Inventory Quantity

```
SQL> SELECT AVG(stock_quantity) AS average_stock FROM Inventory;

AVERAGE_STOCK
-----
33
```

This calculates the **average stock quantity** of all products.